

Shivani S - 192471030

Color a graph with minimum colors while satisfying additional constraints. Clearly define adjacency and constraint conditions. Analyze how constraints increase complexity. Develop a backtracking-based solution. Apply pruning to eliminate invalid colorings. Justify correctness and discuss NP-Complete complexity.

Code:

```
class GraphColoring:
```

```
    def __init__(self, graph, constraints):
```

```
        self.graph = graph
```

```
        self.constraints = constraints
```

```
        self.n = len(graph)
```

```
    def is_safe(self, vertex, color, colors):
```

```
        for neighbor in range(self.n):
```

```
            if self.graph[vertex][neighbor] == 1 and colors[neighbor] == color:
```

```
                return False
```

```
        for u, v in self.constraints:
```

```
            if u == vertex and colors[v] == color:
```

```
                return False
```

```
            if v == vertex and colors[u] == color:
```

```
                return False
```

```
        return True
```

```
    def solve_util(self, vertex, m, colors):
```

```
        if vertex == self.n:
```

```
            return True
```

```
        for color in range(1, m + 1):
```

```
            if self.is_safe(vertex, color, colors):
```

```
    colors[vertex] = color
```

```
    if self.solve_util(vertex + 1, m, colors):
```

```
        return True
```

```
    colors[vertex] = 0
```

```
    return False
```

```
def minimum_coloring(self):
```

```
    for m in range(1, self.n + 1):
```

```
        colors = [0] * self.n
```

```
        if self.solve_util(0, m, colors):
```

```
            return m, colors
```

```
    return None
```

```
graph = [
```

```
    [0, 1, 1, 0],
```

```
    [1, 0, 1, 1],
```

```
    [1, 1, 0, 1],
```

```
    [0, 1, 1, 0]
```

```
]
```

```
constraints = [(0, 3)]
```

```
gc = GraphColoring(graph, constraints)
```

```
min_colors, coloring = gc.minimum_coloring()
```

```
print("Minimum Colors Required =", min_colors)
```

```
for v in range(len(coloring)):
```

```
    print(f"Vertex {v} -> Color {coloring[v]}")
```

Output:

Minimum Colors Required = 3

Vertex 0 -> Color 1

Vertex 1 -> Color 2

Vertex 2 -> Color 3

Vertex 3 -> Color 2

Schedule workflows with dependencies and resource constraints. Clearly define constraints such as task order and resource limits. Analyze how constraints affect scheduling. Develop a backtracking-based solution. Apply pruning techniques effectively. Justify correctness and efficiency.

Code:

```
tasks = {  
    'A': (3, 2, []),  
    'B': (2, 1, ['A']),  
    'C': (4, 2, ['A']),  
    'D': (2, 1, ['B', 'C'])  
}
```

```
limit = 3
```

```
best = float('inf')
```

```
def f(done, t, r):
```

```
    global best
```

```
if t >= best:
```

```
    return
```

```
if len(done) == len(tasks):
```

```
    best = min(best, t)
```

```
    return
```

```
for x in tasks:
```

```
    d, res, pre = tasks[x]
```

```
    if x not in done and all(p in done for p in pre):
```

```
        if r + res <= limit:
```

```
            done.add(x)
```

```
            f(done, t + d, r + res)
```

```
            done.remove(x)
```

```
f(set(), 0, 0)
```

```
print("Minimum Time =", best)
```

Output:

Minimum Time = 11